Markdown Operating System for Robotic Agents (MD-OS CORTEX)

Artificial Prefrontal Cortex (APFC)

Alessandro Rizzo

Independent Research
Correspondence: a.rizzo@physiks.net
ORCID: 0000-0002-8030-3540
Repository: github.com/ciaoidea/MD-OS

Abstract We introduce *Markdown Operating System for Robotic Agents (MD-OS CORTEX)* v5.0. Its embodied principle is simple: the robot acts and sees itself act. From verified command–body–effect relations it learns what it can control, consolidates that operational self-model in its Artificial Prefrontal Cortex (APFC), and uses the model to predict and filter its next actions. The loop converts sensory change into causal self-knowledge and improved action selection; it does not by itself imply consciousness.

Prefrontal networks coordinate goal maintenance, context integration, inhibition, action selection, and error monitoring across distributed brain systems. In this functional and deliberately limited sense, the prefrontal cortex can be compared to an executive operating system; it is not literally a centralized kernel, nor does it contain all perception, memory, or control.

MD-OS CORTEX is an engineering translation of this executive-control role for language-model and robotic agents. *CORTEX* denotes the operational part of the agent: the system that interprets, plans, uses tools, acts, verifies effects, and preserves accepted outcomes. Its evaluated implementation is defined by the composition *CORTEX* = *Codex* + *MD-OS*. Here *Codex* denotes the execution stack, not the persistent identity and not a claim that an LLM is open source. OpenAI's locally running Codex CLI is published as open-source software under the Apache 2.0 license; it exposes files, tools, a terminal, and a bounded read–write–execute loop while mediating a selected OpenAI model that supplies inference. The open-source status applies to the CLI source code, not to the external model weights or hosted service. MD-OS supplies the repository-resident self-frame, method, policies, operational memory, verification, readback, and continuity.

Within MD-OS, the Artificial Prefrontal Cortex (APFC) is the active executive control function, while the Artificial Prefrontal Cortex Graph (APFCG) is its durable heterogeneous representation. APFCG relates Markdown concepts, claims, schemas, and procedures; JSON/NDJSON state and evidence; application workflows and action receipts; deterministic executors; and bounded connectors to APIs and devices. Obsidian exposes its linked-Markdown projection, not the whole graph or execution mechanism. The operational rule is: language proposes; registered executors act; independent readback determines what may be committed. Thus verified experience changes the filter governing future action even before any proposed consolidation into LLM weights.

The supplied artifact passes 194/194 tests and its declared build. In a controlled repair fixture, three candidates pass the visible test, but the verifier admits only the narrow valid repair. In a finite synthetic family, two verified episodes reduce 16 hypotheses to one (4 bits) and change sealed holdout success from 0/2 to 2/2 under equal budgets. These bounded results support method-bearing verified operation, not general intelligence. A future extension proposes offline "robotic REM" replay into an isolated LLM clone or adapter, followed by sealed promotion and reduced APFC intervention during verified flow. Parameter consolidation, biological equivalence, AGI, host security, and physical-robot safety are not demonstrated; in robotics, an independent real-time safety controller retains final veto authority.

Keywords: language-model agents, artificial prefrontal cortex, robotic agents, sensorimotor self-model, operational self-perception, operating filesystem, supervisory control, verification, cumulative learning

1 Introduction

Philip K. Dick asked, *Do Androids Dream of Electric Sheep?*,[1] the novel that inspired *Blade Runner*. The engineering question here is narrower: what should a robotic agent replay while offline so that it returns with a better capability rather than merely a longer memory? MD-OS answers: only episodes bound to action, outcome, evidence, and correction; only a candidate that still passes new-task, regression, identity, and safety tests may alter future behaviour.

Suppose an agent is asked to fix a program. It edits a file and says, "The bug is fixed." What has actually been established? Only that the model produced a sentence. The sentence may be correct, but it is not the evidence. A useful operating system for agents needs a more physical answer:

1. Which file changed?
2. Was the action allowed?
3. Did the intended test pass?
4. Did an independent test also pass?
5. Were valid behaviors preserved?
6. Can another process reconstruct what happened?

There is a second problem. A capable model can solve a difficult step in a burst and then lose the result, repeat the investigation, or regress on the next step. Peak performance and retained progress are different quantities. A method matters because it turns some successful episodes into reusable, verified

structure. It does not manufacture intelligence; it determines how much useful work survives.

MD-OS CORTEX addresses both problems by moving operational context out of an ephemeral conversation and into typed, inspectable artifacts. Markdown carries human-correctable identity, knowledge, policy, and procedure; JSON and NDJSON carry task specifications, receipts, event streams, and generated state; deterministic programs build and replay derived views. The paradigm is *Operational Context as Filesystem*, not Markdown as an isolated format.

The term *Artificial Prefrontal Cortex* names a biologically inspired functional abstraction of selected executive operations: context integration, goal maintenance, inhibition, action selection, and error monitoring. The analogy is architectural, not anatomical: the implementation makes no claim of neural tissue, consciousness, cellular dynamics, or one-to-one equivalence with the human prefrontal cortex. Likewise, transient hypofrontality is a hypothesis about selective reconfiguration during skilled flow, not the shutdown of the entire PFC. Imaging results report both regional decreases and increases during flow;[2–5] they motivate a control question but do not specify software.

The paper therefore has two levels. The evaluated thesis is narrow:

> *For bounded tasks, a file-native supervisory layer can make a successful agent operation depend on explicit task contracts, registered capabilities, execution receipts, postcondition checks, and replayable evidence rather than on the agent's verbal assertion.*

The broader, testable thesis is that durable capability depends on both model capacity and method. A well-governed weaker model may outperform an unaided stronger model on persistent multi-step work if it preserves more verified progress. This is not a universal weak-over-strong claim, and the present artifact does not test it across models. The distinction is essential: the paper evaluates mechanisms that could make progress cumulative; it does not declare the comparative hypothesis proved.

The thesis is decomposed into auditable claims in table 1. Deductive claims are derived from inspected code under the assumptions in section 6; observed claims are reproduced on the official GitHub baseline; design claims are specified but not empirically validated here.

1.1 Contributions

The paper contributes:

1. a compact model connecting stable self-reference, world events, operational meaning, experience, and durable episodes;
2. a closed sensory–agentic mechanism in which verified visual self-observation consolidates a body–action–effect model that APFC uses to filter future actions;
3. an operational semantics separating proposal, authorization, execution, verification, and durable commitment;
4. four implementation-level propositions with stated assumptions and proofs;
5. a reproducible implementation audit covering static checks, 194 tests, the complete build chain, a verifier adversarial fixture, bounded learning, and replay;
6. a claim-to-evidence matrix that records negative results as first-class outcomes;
7. a falsifiable distinction between raw capacity and cumulative verified progress, without presenting it as an observed cross-model result; and
8. a robotics integration architecture that preserves an independent hard safety boundary.

The original project schema has been redrawn in figure 1 without changing its central claim. The running cortex in this release is *Codex + MD-OS*. The Codex execution stack couples the open-source local CLI to a selected external model and its authorized tools; MD-OS is the dynamic, persistent operational structure that makes the loop method-bearing. Sensors and connectors bring events into that structure. Tasks, memory, and policies give those events context. Semantic actions return bounded effects to the world. Receipts establish what happened, verifiers determine what succeeded, episodes preserve the result, and replay determines what may influence later action.

2 Relation to Prior Work

Language-model agents commonly interleave reasoning and action. ReAct couples reasoning traces with external actions;[6] Reflexion stores verbal feedback in episodic memory without changing model weights;[7] Generative Agents combine observation, reflection, and planning in a persistent simulated environment;[8] MemGPT uses operating-system-inspired memory tiers to extend effective context;[9] and Voyager accumulates an executable skill library from environment feedback.[10]

OpenClaw is a particularly important comparator because it already combines a self-hosted gateway, agent sessions, workspace bootstrap files, memory, tools, skills, plugins, and automation.[11–13] It therefore rules out a weak novelty claim: local files plus tool calls do not, by themselves, make MD-OS new. OpenClaw's documented surface is primarily a host runtime that makes agents reachable and capable. MD-OS is proposed at a different abstraction: a host-portable operational method that defines what must remain true before, during, and after an action. This is a comparison of published contracts, not a benchmark and not a claim that OpenClaw could never be configured to implement similar controls.

The concrete Codex–MD-OS composition. The evaluated v5.0 artifact does not run by itself. "Codex" must first be unpacked into its distinct parts. **Codex CLI** is OpenAI's local coding agent: its source is publicly developed in OpenAI's official `openai/codex` GitHub repository, which is released under the Apache 2.0 license.[14,15] The CLI gives a selected model a local working environment, file operations, an integrated terminal, tool use, and repository instructions such as `AGENTS.md`.[16] **The OpenAI model** is the inference engine reached through that host. Publishing the CLI source does not publish the model weights and does not make the hosted model service open source. In this paper, *Codex* is therefore a shorthand for the operational stack used in the experiment:

Table 1. Claim ledger. This table defines the boundary of the paper's affirmative claims.

ID	Claim	Status	Evidence
C1	Declared task, evidence, and observation paths are canonically confined to `md-os/`, absent a check-use race.	Deductive	Proposition 1 plus negative tests.
C2	A task specification cannot directly provide an arbitrary shell argument vector; it selects a registered command identifier.	Deductive	Proposition 2.
C3	`verified` implies the verifier found no missing acceptance test, policy block, incomplete receipt, required-delta failure, acceptance-test failure, or required-evidence failure.	Deductive	Proposition 3.
C4	An event mutation is detected unless every affected hash, sequence number, and downstream link is consistently recomputed.	Deductive	Proposition 4.
C5	The supplied verifier rejects two plausible false positives and accepts the narrow valid patch in one controlled fixture.	Observed	Section 8.2.
C6	A finite four-constraint rule transfers from two development cases to two sealed cases in the supplied experiment.	Observed, narrow	Section 8.3.
C7	The evaluated state reaches a replay fixed point after evidence integration.	Observed, snapshot-only	Section 8.4.
C8	APFC can be placed above a robot safety/controller layer as a high-level filter and coordinator.	Design only	Figure 5; no robot experiment.
C9	Webcam self-observation can supply command-dependent bodily evidence for a learned APFC future-action filter.	Demonstration plus testable mechanism	Section 4.6; quantitative learning and ablation remain to be run.

Table 2. Architectural distinction from the documented OpenClaw host surface. The difference is not "can the agent act?" but "what makes an action a verified, learnable operation?"

Question	OpenClaw documented surface	MD-OS CORTEX/APFC contract evaluated here
Primary unit	Agent turn, session, routed message, tool, skill, or automation	Validity-bearing operation and its durable evidence chain
Persistent context	Workspace/bootstrap files, memory, session state, and skills	Typed self, task, policy, capability, receipt, verifier, episode, and graph artifacts
Completion	Defined by the active agent workflow, tool result, and skill instructions	`verified` requires explicit conditions; unsupported success is not committed
Learning method	Memory and reusable skill surfaces are available to the agent	Episode, reproduction, holdout, no-regression, promotion, replay, and rollback are separate gates
Host relation	Integrated gateway and agent runtime	Operating context intended to survive a change of compatible host runtime

the open-source local CLI plus the selected external OpenAI model and its authorized tool interfaces.

MD-OS is a separate layer. It programs how that capacity is oriented, constrained, checked, remembered, and resumed. It carries the persistent self-frame, APFC executive method, APFCG, policy, episodic records, verification, and readback. The complete running object in this study is therefore

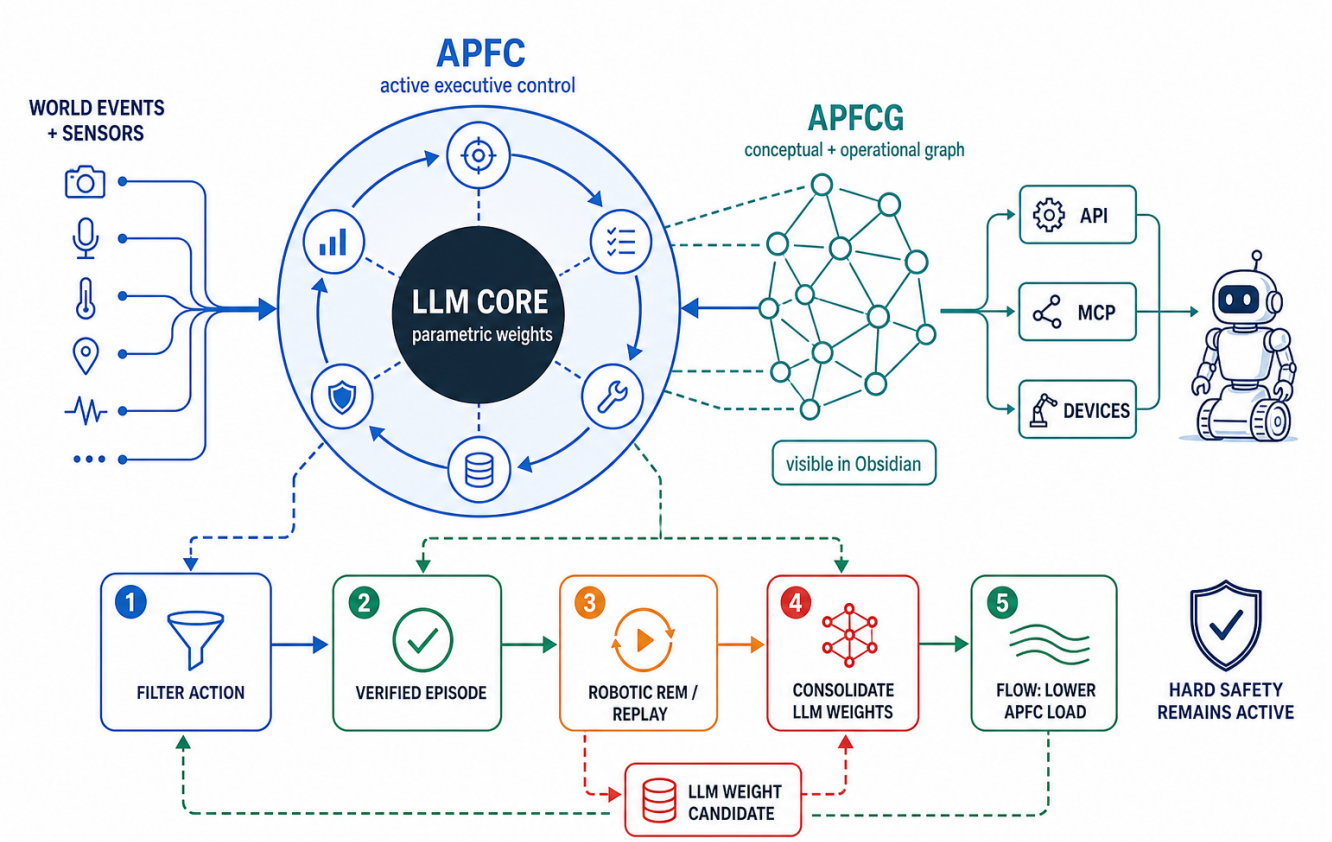
$$\mathrm{CORTEX}_{\mathrm{run}} = \underbrace{\text{Codex}}_{\substack{\text{Apache-2.0 CLI}\\ \text{external model+tools}}} + \underbrace{\text{MD-OS}}_{\substack{\text{APFC+APFCG}\\ \text{identity+method+readback}}} . \quad (1)$$

This equation describes a composition, not ownership or identity equivalence. The Codex CLI is open-source host software; the model is the inference substrate; MD-OS is the repository-resident persistent identity and method. None of these statements makes one layer the source code, weights, or identity of another. Another host is only a compatibility claim after its bootstrap, permissions, command behaviour, and runtime readback have been verified.

MD-OS CORTEX occupies a different layer. It does not propose a new foundation model or claim a new reasoning algorithm. It is a persistent control and evidence plane around a model. The closest practical comparison is an agent-computer interface: SWE-agent shows that the interface presented to a language model materially affects software-engineering behavior.[17] AgentBench and OSWorld similarly emphasize interactive execution and environment-grounded evaluation.[18,19] The present work asks a narrower question: what must be recorded and checked before a particular operation may be called successful?

The artifact's provenance representation is consistent in spirit with the W3C PROV distinction among entities, activities, agents, and derivations.[20] Its explicit risk and evidence boundaries also align with the general direction of the NIST Generative AI profile, which treats governance, measurement, and management as lifecycle activities.[21] We do not claim formal conformance to either standard.

For robotics, ROS 2 provides middleware and production-oriented communication infrastructure;[22] control barrier functions exemplify mathematically grounded mechanisms for enforcing safety properties at the controller level.[23] APFC is not a substitute for either. It belongs above them as a non-real-time mission and evidence coordinator.



CORTEX = CODEX + MD-OS. Codex here means the execution stack: OpenAI's local Codex CLI is open-source Apache 2.0 software (`github.com/openai/codex`) and mediates a selected external OpenAI model, tools, terminal, and bounded read–write–execute loop. The CLI source is open; this does not claim that model weights or hosted services are open source. MD-OS supplies the persistent self-frame, executive method, APFC control, APFCG, policy, memory, verification, and readback.

EMBODIED LEARNING: ROBOT ACTS → WEBCAM OBSERVES THE ROBOT → APFC LEARNS BODY–ACTION–EFFECT → APFC FILTERS THE NEXT ACTION.

Fig. 1. MD-OS CORTEX and the proposed wake–consolidation–flow cycle, redrawn from the original project schema. The central LLM weights are not the whole agent. In the implementation evaluated here, the functioning cortex is the composition of the Codex execution stack—the open-source local CLI plus a selected external model—and MD-OS, the persistent APFC operating filesystem. APFC is the active executive function; APFCG is its durable heterogeneous graph. APFCG links Markdown pages, concepts, claims, and schemas; JSON/NDJSON state and evidence; application workflows and action receipts; Python, JavaScript, PHP, shell, and other bounded executors; and connectors to APIs and devices. Obsidian visualizes the linked-Markdown projection, not the whole graph or execution mechanism. Because the robot is itself present in the sensory field, webcam feedback is also self-observation: verified command-dependent bodily changes consolidate an operational self-model that APFC uses to predict and filter later actions. The lower cycle—robotic replay, candidate weight consolidation, sealed promotion, and reduced APFC load during verified flow—is proposed; hard safety remains active.

3 Neuroscientific Grounding and Engineering Translation

3.1 The prefrontal cortex as an executive control plane

The useful analogy is precise but limited. The prefrontal cortex can be understood as an executive operating system for behaviour: it does not itself perform all perception, memory, emotion, or movement, just as a computer operating system does not itself perform every application computation. Instead, prefrontal networks coordinate distributed resources by maintaining goals and context, directing attention, inhibiting unsuitable responses, selecting actions, monitoring errors, and reorganizing behaviour when conditions change. In this functional sense, the PFC is a control plane over many specialized processes.

It is not literally a centralized biological kernel. Control is distributed, recurrent, and inseparable from interactions with hippocampal, thalamic, sensory, motor, limbic, and other cortical systems. The defensible translation is therefore not "PFC equals Windows", but

$$\text{PFC} : \text{brain} \approx \text{APFC} : \text{CORTEX}_{\text{run}}, \tag{2}$$

where the relation denotes an executive-control role, not anatomical or mechanistic identity. In the present artifact, $\text{CORTEX}_{\text{run}}$ = Codex + MD-OS: Codex supplies active cognitive-execution capacity, while MD-OS/APFC schedules, constrains, routes, verifies, and records its operational processes.

3.2 Executive control is not memory storage

The biological starting point must be stated correctly. The prefrontal cortex does not contain a notebook of all experience, and the hippocampus is not a simple disk. Prefrontal and hippocampal systems have distinct, interacting roles: prefrontal networks help maintain goals and context, select relevant memories, and organize behavior; hippocampal networks rapidly

Table 3. Functional translation from neuroscience to MD-OS. Each row states a useful engineering correspondence and its boundary.

Biological function	MD-OS mechanism	Boundary of the correspondence
Rapid contextual encoding	Verified episode binds event, task, action, evidence, and outcome	A file record is not a hippocampal engram
Offline reactivation	Bounded readback replays selected successes, failures, and surprises	Builder execution is not an oscillatory sleep process
Systems consolidation	Repeated evidence restructures the APFC/knowledge graph and candidate skills	Graph rewrite changes external operational memory, not LLM synapses
Slow cortical integration	A future training stage distils graph-supported relations into model or adapter weights	Parameter learning requires an explicit optimizer, data, compute, and tests
Selective forgetting/downscaling	Contradictions, noise, superseded rules, and low-value detail are demoted or archived under provenance rules	Deletion is not assumed beneficial and remains auditable
Skilled automaticity	Verified low-risk skills may run with less deliberative APFC intervention	Identity, policy, anomaly detection, and hard safety remain active

bind events to their spatial and temporal context.[24] Long term memory depends on interactions among hippocampal, cortical, thalamic, and other systems. Consequently, MD-OS CORTEX/APFC is not a model of one anatomical region. It is an engineered executive layer coupled to an explicit episodic store and a slower knowledge/skill layer.

This correction strengthens rather than weakens the architecture. It explains why `ME.md`, APFCG, episode records, deterministic builders, and an LLM have different jobs. The self-reference anchors goals and limits; the APFC schedules and gates; episodes preserve particulars; the graph organizes relations; the LLM supplies distributed generative competence. Calling all of these "the artificial PFC" would hide the mechanism that the analogy is meant to clarify.

3.3 What sleep consolidation contributes

Sleep is not passive storage. A major account of systems consolidation holds that recently encoded patterns are selectively reactivated during offline periods. During non-REM sleep, hippocampal sharp-wave ripples, thalamocortical spindles, and cortical slow oscillations are temporally coordinated; repeated replay can gradually transform and integrate representations in cortical networks.[25,26] The result is not a bit-for-bit copy. Episodic detail may be retained, weakened, related to older knowledge, or abstracted into a more general structure. REM sleep may contribute complementary plasticity and stabilization, but a simple "NREM transfers, REM finishes" chain remains a model rather than a settled complete mechanism.[25,26]

The computational lesson is the coupling of a fast episodic system and a slow integrating system. Neural-network models of complementary learning systems show why regulation matters: indiscriminate transfer can memorize noise and harm generalization, whereas replay selected for predictability can improve generalization.[27] Artificial-network experiments also show that replay can reduce catastrophic forgetting, although scaling and domain transfer remain open problems.[28] These results support replay as an engineering principle; they do not prove that a Markdown runtime reproduces sleep physiology.

3.4 APFCG in Obsidian and from graph to model weights

Obsidian displays links among Markdown files; it does not display a neural network and cannot inject those links directly into an LLM. The useful object is the underlying typed APFCG: claims, goals, procedures, evidence, counterexamples, episodes, and their relations. To affect model weights, that graph must be compiled into training signals. Let G be the verified graph and E the accepted episode set. A deterministic consolidation compiler produces a dataset

$$\mathcal{D}_G = C(G, E), \tag{3}$$

containing input–target pairs, preference pairs, counterexamples, preserved skills, and policy-sensitive negative cases, each linked back to its source nodes. A cloned model or adapter with parameters θ' can then be trained with a mixed objective such as

$$\begin{aligned}\theta' = \arg\min_{\phi} \big[& \mathcal{L}_{\text{new}}(\mathcal{D}_G; \phi) \\ & + \lambda \mathcal{L}_{\text{replay}}(\mathcal{D}_{\text{old}}; \phi) \\ & + \mu \mathcal{L}_{\text{safety}}(\mathcal{D}_{\text{safe}}; \phi) \big].\end{aligned} \tag{4}$$

The replay and safety terms matter because fitting only new episodes invites catastrophic forgetting. Parameter-efficient adapters such as LoRA provide one practical candidate mechanism, not the only one.[29]

The candidate does not replace the active model merely because training loss fell. Promotion requires sealed new-task evaluation, old-skill regression, safety and permission tests, identity-drift checks, provenance, and rollback. Only a candidate that improves verified generalization without unacceptable loss is promoted. APFCG remains the auditable source record; weights are a compressed execution substrate, never the sole copy of the method. This graph-to-weights stage is a proposed extension, not an implemented or evaluated feature of v5.0.

4 System Model

4.1 APFC and its persistent APFCG

APFC names the active executive function: it interprets events relative to the agent, maintains goals and constraints, routes admitted actions, checks their effects, and decides which verified outcomes may influence later behaviour. APFCG is the persistent file-native graph on which that function operates. A minimal decomposition is

$$G_{\text{APFC}} = (V_c \cup V_o,\ E_c \cup E_o \cup E_x), \tag{5}$$

where V_c, E_c are conceptual nodes and relations, V_o, E_o are operational nodes and transitions, and E_x binds meaning to action and action outcomes back to knowledge. The conceptual set V_c includes Markdown pages, concepts, claims, schemas, counterexamples, and explanatory relations. The operational

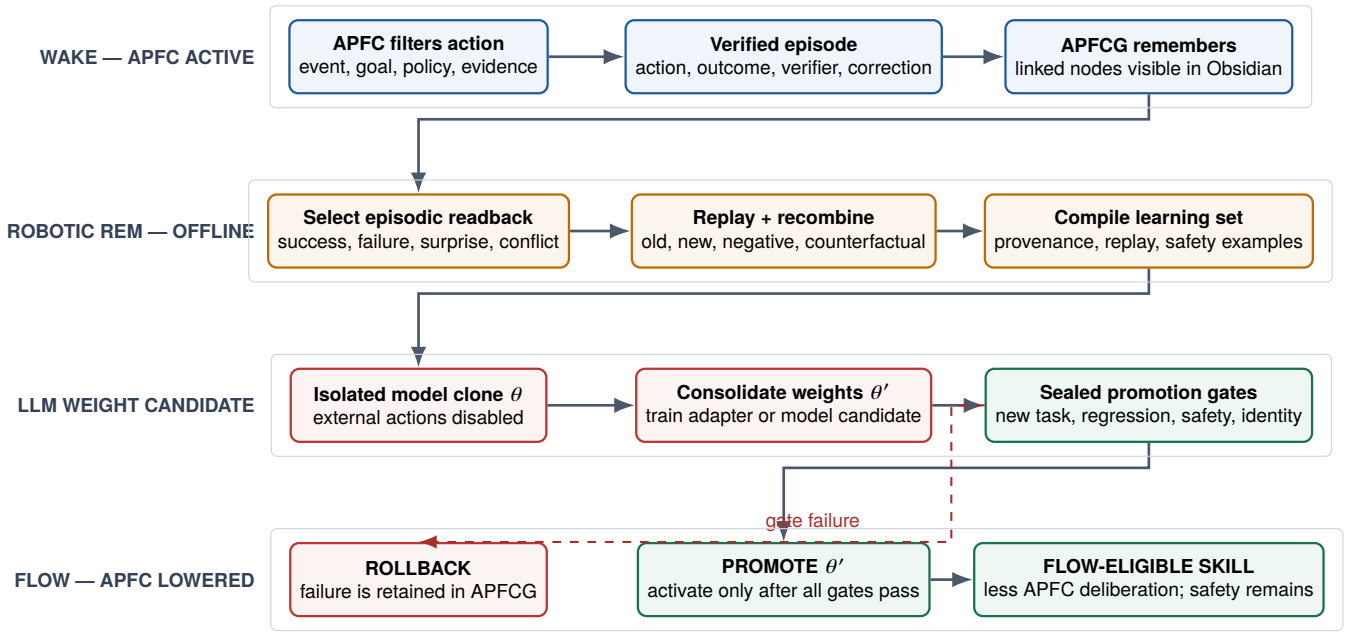


Fig. 2. The proposed wake–robotic-REM–consolidation–flow cycle. During wake, APFC filters action and commits verified outcomes to APFCG. A bounded offline phase, called *robotic REM* here, selects episodic readback, replays and recombines evidence, and compiles provenance-linked learning sets. An isolated LLM clone or adapter is trained and promoted only after sealed new-task, regression, safety, and identity gates. A consolidated skill may then require less deliberative APFC intervention, while identity, anomaly detection, evidence capture, and hard robot safety remain active. "Robotic REM" is an engineering name for this offline process, not a claim that the software reproduces biological REM sleep. Parameter consolidation and flow transition are proposed extensions, not reported v5.0 results.

set V_o includes JSON/NDJSON state and evidence, application workflows, connector calls, action receipts, and bounded executors implemented in Python, JavaScript, PHP, shell, or another registered language. A Markdown node may point to a schema or executable, but natural language never grants authority by itself. Linked Markdown files make the graph human-readable and visible in Obsidian. Typed state, schemas, and deterministic builders make the same graph executable, checkable, and reconstructible. Obsidian can therefore display APFCG, but it is neither APFCG's sole storage nor the APFC that operates on it.

4.2 A relative origin for experience

An event does not carry its operational meaning by itself. A low battery can be irrelevant to a stationary agent and decisive to an agent halfway through a delivery. Meaning depends on the event, the current goal, the available capabilities, the policy, and the agent that must continue after the event.

Let the operational self at time t be

$$S_t = (M, K_t, W_t, G_t, P_t), \tag{6}$$

where M is the stable self-reference anchored by `ME.md`, K_t is durable knowledge, W_t is volatile working context, G_t is the active goal set, and P_t is policy. For a world event e_t, the APFC binding function produces

$$(x_t, z_t) = B(e_t, S_t), \tag{7}$$

where x_t is the resulting experience record and z_t is its operational meaning: what, if anything, the agent should preserve, inhibit, investigate, or act upon. This is a computational definition, not a claim about subjective phenomenology.

The role of `ME.md` is precise. It gives the system a stable relative origin, so it need not reconstruct "who is acting and under which limits" from every new prompt. It does not freeze the agent. Working context, knowledge, goals, and skills may change, while identity-level changes remain explicit and auditable. Without that separation, a transcript is easily mistaken for a self and an event log is easily mistaken for experience.

4.3 Five objects, not one conversation

Definition 1 (Operational state)**.** At time t, the supervisor state is $X_t = (K_t, W_t, P_t, C_t, L_t)$, where K_t is durable knowledge, W_t is active working state, P_t is policy, C_t is the registered capability set, and L_t is the evidence ledger.

Definition 2 (Bounded task)**.** A task is $T = (g, A, Q, O, E, B)$, where g is the goal, A is the declared action set, Q is the set of executable acceptance tests, O is the observation-target set, E is the required evidence, and B is the risk and resource budget.

Definition 3 (Action receipt)**.** For action $a \in A$, the executor produces a receipt $r(a) = (h_a, t_0, t_1, s_{\text{exit}}, H_{\text{pre}}, H_{\text{post}}, \Delta, \mathcal{A})$, binding the normalized action hash h_a, timestamps, exit status, pre- and post-state hashes, observed delta Δ, and produced artifacts $\mathcal{A}$.

Definition 4 (Verification outcome)**.** The deterministic verifier returns

$$V(T, R) \in \{\texttt{verified}, \texttt{failed}, \texttt{unverified}\},$$

where R is the set of action receipts. A policy block or a critical failed check yields `failed`; an attention-level unresolved condition yields `unverified`; only the absence of both yields `verified`.

Definition 5 (Committed operation)**.** An operation is complete only when the following chain has closed:

$$\text{RequestValid} = E_p \wedge S \wedge P, \tag{8}$$

$$\text{ExecutionValid} = \text{RequestValid} \wedge C \wedge A_u \wedge B_x, \tag{9}$$

$$\text{OperationValid} = \text{ExecutionValid} \wedge E_x \wedge V \wedge L, \tag{10}$$

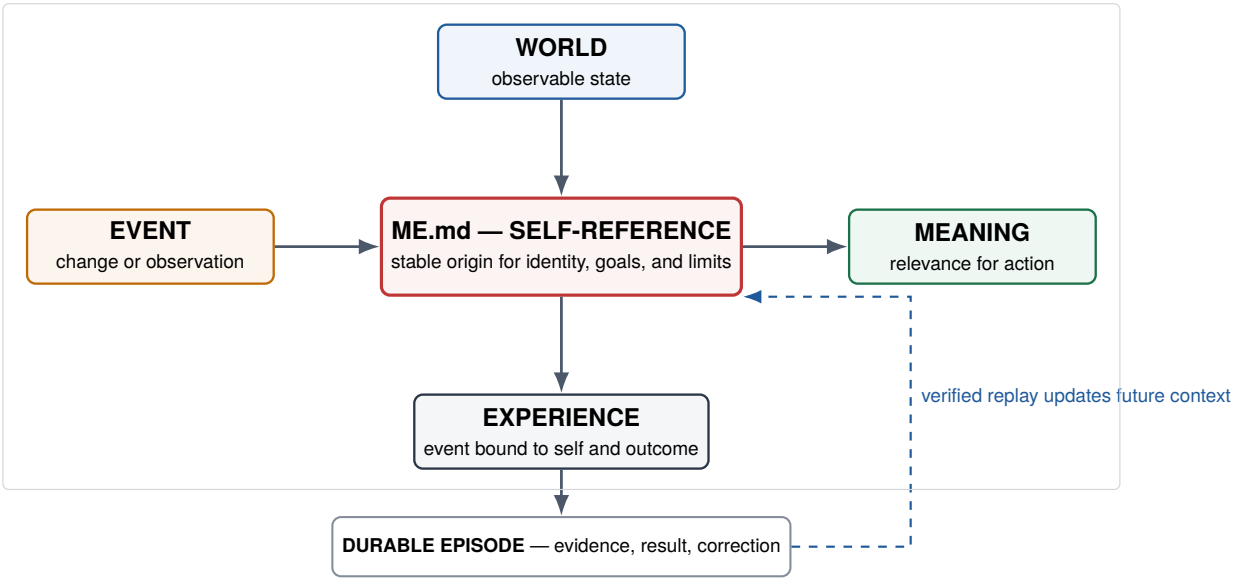


Fig. 3. The APFC self-reference crossing. The world supplies observable state and events. `ME.md` supplies the stable relative origin from which an event acquires operational meaning and becomes an experience. A verified experience is committed as an episode and may update future context. `ME.md` anchors the self; it is not the whole dynamic working state.

where E_p is epistemic classification, S semantic/task validity, P policy admission, C capability existence, A_u required authorization, B_x brokered execution, E_x an execution receipt, V postcondition verification, and L durable ledger commitment.

This equation is an architectural contract. The v5.0 implementation mechanizes important edges of the chain; it does not provide a hardware reference monitor proving every host effect.

4.4 File-native composition

The operational pipeline is:

$$\begin{aligned}\text{intent} &\rightarrow \text{TaskSpec} \rightarrow \text{registered action} \rightarrow \text{ActionReceipt} \\ &\rightarrow \text{Verification} \rightarrow \text{Episode} \rightarrow \text{replayable state.}\end{aligned}$$

The implementation separates source and projection:

- Markdown files document goals, policies, procedures, and human-readable state.
- JSON files carry schemas, task specifications, receipts, verification records, graphs, and indexes.
- NDJSON files carry append-oriented journals and APFC events.
- Deterministic Node.js builders regenerate summaries and indexes.

This separation matters because a generated summary is not the authority for its own truth. Canonical inputs can be inspected; derived views can be rebuilt; and a verifier can refer to files rather than to hidden conversational memory.

The self-reference crossing and the execution pipeline describe different parts of one loop. The first explains how an event becomes relevant to this agent and this goal. The second explains how that relevance becomes a bounded action and, if verified, a durable episode. Replay closes the loop by making the evidence available to later tasks without rewriting history into success.

4.5 Robotics placement

The biologically inspired placement of APFC in the frontal region of an embodied agent expresses a functional relation: sensory evidence must be integrated with goals and constraints before it becomes coordinated action. Figure 5 makes that information flow explicit. APFC has four supervisory duties: classify evidence, maintain task context, inhibit or sequence high-level proposals, and compare expected with observed outcomes. It has no direct motor authority. The independent safety layer in the torso can veto an action even after APFC allows it.

4.6 The closed sensory–agentic loop

The robot acts. The webcam sees the robot's body change. MDOS compares that change with the issued command, verifies whether the relation repeats, and stores the result in APFCG. APFC then uses the learned relation to decide what the robot should or should not do next. This is the causal loop:

$$\boxed{\begin{aligned}&\text{act} \rightarrow \text{observe self} \rightarrow \text{attribute effect} \rightarrow \text{verify} \\ &\quad \rightarrow \text{consolidate APFC} \rightarrow \text{filter next action.}\end{aligned}} \tag{11}$$

When a robot issues a bounded motor probe and then observes the resulting change through a webcam, the sensory stream contains evidence about both the world and the robot as an object within that world. Repeated covariation between a command and a visual or proprioceptive change can establish a sensorimotor contingency: which controllable body part changes, in which direction, over what delay and range, with what uncertainty and risk. This is consistent with sensorimotor accounts of perception and with evidence for predictive internal models in motor control.[30,31]

Let s_t be the current operational self-state, a_t a bounded command, and o_{t+1} the next observation. A learned forward model M_t predicts

$$\hat{o}_{t+1} = M_t(s_t, a_t), \tag{12}$$

$$\epsilon_t = d(o_{t+1}, \hat{o}_{t+1}), \tag{13}$$

$$M_{t+1} = U(M_t, s_t, a_t, o_{t+1}, \epsilon_t, v_t), \tag{14}$$

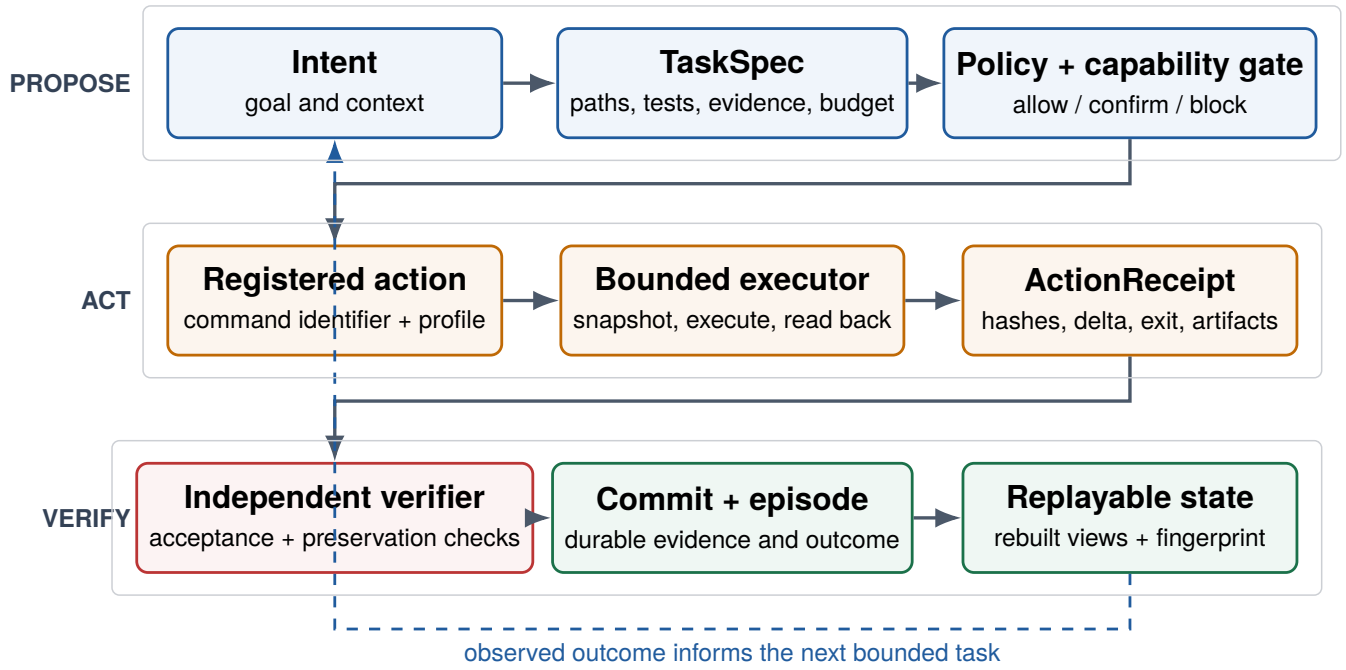


Fig. 4. MD-OS APFC operational pipeline. A proposal becomes a committed operation only after policy and capability admission, bounded execution, an explicit receipt, and independent verification. Replay rebuilds derived views from durable evidence; it does not convert an unsupported verbal claim into a verified result.

where d measures sensory prediction error and v_t is verifier readback. Only verified, repeatable contingencies are admitted to the persistent self-model. The resulting model is not merely a memory of past movement. It changes the APFC action-admission function:

$$\mathcal{A}_t^{\mathrm{APFC}} = \{a \in \mathcal{A} : P(a) \wedge C(a), \; R(M_t(s_t, a)) \le \rho, \; Q(M_t, s_t, a) \ge q_{\min}\}, \tag{15}$$

where P is policy admission, C capability, R predicted risk, and Q confidence in the learned action–effect relation. Before learning, an arm command may have an unknown consequence. After consolidation, APFC can reject commands whose expected effect is unsafe or unreliable, choose commands whose predicted effect advances the goal, and detect a fault when observation diverges from prediction. Learning has therefore become a filter over future action, not an annotation of past action.

This mechanism produces a limited but genuine form of *operational self-perception*. The robot represents itself as a persistent causal object with controllable parts, current state, action-dependent effects, limits, and affordances. It can distinguish "a commanded change in my arm" from an unexplained change in the scene and use that distinction prospectively. This is not a claim of phenomenal consciousness or human self-awareness. It is a functional self-model with measurable consequences for prediction, planning, error detection, and control. Visual self-models have independently been shown to support robot motion planning and control, providing a concrete comparison class.[32]

Demonstration artifact. The video *Robotic Lab—Speedy Evolv learns to use its new arm attachment by observing itself on webcam* documents the intended physical loop: the robot acts through the arm and receives webcam observations of its own resulting motion.[33] Its architectural importance is that the camera is not merely monitoring the robot for a human viewer; it supplies the reafferent evidence from which MD-OS CORTEX can bind command, bodily change, error, and correction into an APFC episode. Consolidation first occurs in the file-native APFC/APFCG as a verified action schema and forward model. A later robotic-sleep stage may distil selected relations into LLM or adapter weights, but weight change is not required for the learned APFC filter to alter the next action.

The video is a demonstration, not by itself a controlled learning result. A strong evaluation must log synchronized command–observation pairs, show lower held-out prediction error after exploration, improve task success or safety on new arm targets, preserve the model across a cold start, and remove the gain when the consolidated APFC self-model is ablated. We call the resulting property *operationally emergent* only if it was not supplied as a direct command–effect table, arises through the closed loop, persists as reconstructible state, improves sealed future behaviour, and is lost under ablation. Under this definition, greater intelligence means a measured increase in predictive and action-selective competence; it does not mean an untested assertion of consciousness.

5 Implementation

5.1 Inspected snapshot

The evaluated artifact is the official ciaoidea/MD-OS repository baseline at commit `8e988b7f3fa677993afa5040ff8534002b1bc83e`. Its `package.json` identifies package version 5.0.1, the `GPL-2.0-only` license, and Node.js ≥ 20. Evaluation used Node.js 24.19.0 on Linux 6.18.35 x86-64. The verified Git and npm package surfaces contained no absolute host paths, credentials, private keys, or local runtime cache; host-local operational state remained outside the published source.

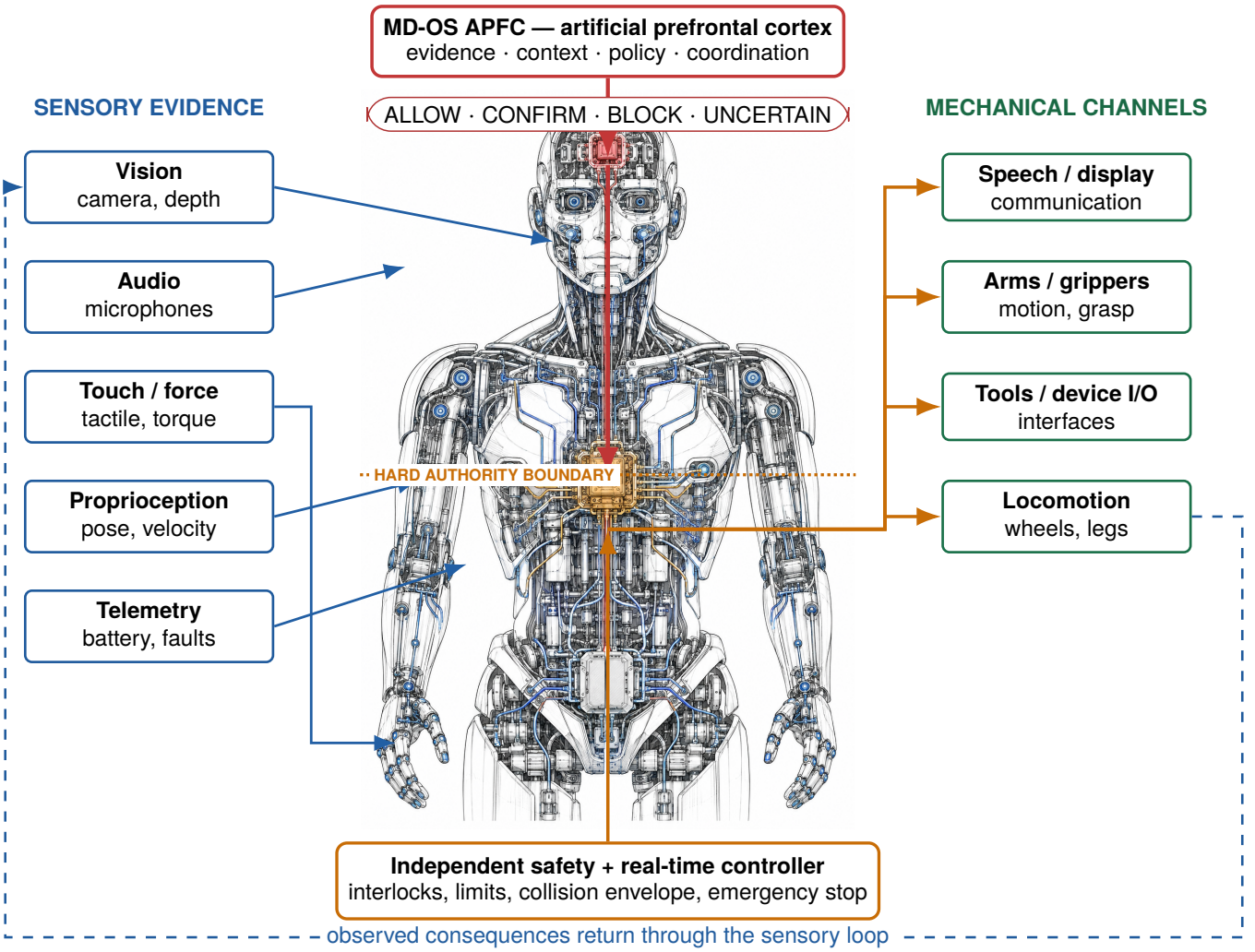


Fig. 5. APFC as the action-filtering layer of an embodied robotic agent. The frontal placement is a biologically inspired functional mapping of selected executive roles: multimodal evidence is integrated with context, goals, and policy before a mission-level request is released. The request then crosses a hard authority boundary into an independent safety and real-time controller. Only that downstream controller commands mechanical interaction channels. The mapping is not a claim of anatomical or neural equivalence, and the figure is an interface design rather than a reported hardware experiment or certification.

5.2 Code and artifact availability

The canonical MD-OS source repository is `https://github.com/ciaoidea/MD-OS`. The editable manuscript is stored at `docs/papers/zenodo/paper.tex`; its reviewed PDF, bibliography, figures, and SHA-256 manifest are stored in the same Zenodo package directory. The covered repository material is distributed under `GPL-2.0-only`; cited and third-party material retains its own terms.

5.3 Task and connector boundary

A task may name only the `terminal_executor` connector in the inspected cognitive transaction compiler. It supplies a safe `project_id` and `command_id`; the terminal connector then looks up the corresponding argument vector in its registered profile. The task does not pass a raw shell string to `spawnSync`. The child process receives only the host `PATH` environment variable, a configured timeout, bounded output buffers, and configured redaction markers.

This is useful input reduction, but it is not a complete security boundary. A writer who can alter the command profile or runtime code can alter the allowed behavior. The host operating system remains the ultimate enforcement layer.

5.4 Receipts and verification

For each action, the executor snapshots every declared observation target before and after execution. Files are hashed by content; directories by a bounded manifest; symbolic links are represented as unsafe targets. The receipt records the normalized action hash, timestamps, expected and observed exit states, artifacts, state hashes, deltas, rollback metadata, and connector readback.

The verifier is separate from the planner in code path and decision rule, but not in an adversarial security domain. It accepts a task only after checking the declared executable criteria. In the software-repair benchmark, an oracle outside each candidate worktree provides a stronger independent check.

5.5 Replay and projections

Replay computes a semantic fingerprint from generated state, runs the builder sequence, and compares the new fingerprint. This is more informative than byte-for-byte equality because timestamps and journals can change without changing the intended semantic state. It is nevertheless an implementation metric, not a proof that all external effects are deterministic.

Table 4. Implementation elements used by the paper's claims.

Mechanism	Principal implementation	Role
Path canonicalization	`md-os/os/lib/common.js`	Resolves the existing ancestor through `realpath`, reconstructs missing segments, and rejects a relative path that escapes its root.
Task compilation	`md-os/kernel/cognition/task_compiler.js`	Normalizes task paths, safe identifiers, budgets, registered commands, acceptance tests, observation targets, and evidence requirements.
Bounded execution	`md-os/kernel/cognition/executor.js` and `md-os/os/terminal_connector.js`	Selects a pre-registered command, snapshots declared targets, executes, and writes an action receipt.
Postcondition verification	`md-os/kernel/cognition/verifier.js`	Runs acceptance checks, verifies receipt completeness, required deltas, and required evidence; computes a three-valued verdict.
APFC event ledger	`md-os/apfc/executive/event_recorder.js`	Validates event shape, phase order, sequence, payload hash, event hash, and previous-event hash.
Replay	`md-os/os/replay_runtime.js`	Runs deterministic builders and compares a semantic fingerprint before and after replay.

6 Assumptions and Formal Properties

6.1 Trust assumptions

The propositions below are conditional on the following assumptions.

A1 Node.js, the host kernel, filesystem primitives, and `spawnSync` implement their documented semantics.

A2 SHA-256 is collision resistant for the evaluated records, and canonical JSON serialization is deterministic.

A3 Runtime code, connector profiles, task specification, and verifier definitions are not maliciously rewritten during one transaction.

A4 Declared acceptance tests and independent oracles correctly encode the bounded specification they claim to test.

A5 No attacker changes a relevant symbolic-link topology between a canonical path check and its subsequent use.

A6 For the benchmark result, the independent oracle remains outside candidate worktrees and the diff policy remains enforced.

These assumptions expose the boundary between an evidence-oriented supervisor and a secure host reference monitor. They are not hidden qualifications.

6.2 Path confinement

Proposition 1 (Canonical confinement of declared task paths)**.** *Under **A1**, **A3**, and **A5**, every task specification path, required-evidence path, and observation-target path returned by* `normalizeMdosPath` *denotes a location inside the canonical* `md-os/` *root.*

Proof. The compiler first resolves the candidate against the workspace. Function `assertInsideRoot` canonicalizes the root and the nearest existing ancestor of the target with `realpath`; it then reattaches any missing suffix. Let r be the canonical root and p the resulting canonical candidate. The implementation computes $d = \text{relative}(r, p)$ and rejects when d is `..`, starts with `../`, or is absolute. For accepted d, path normalization therefore contains no component that leaves r, and the returned location is r/d. Existing symbolic links are already resolved in p, so an existing link to an exterior location makes d escape and is rejected. **A5** is required because this check does not eliminate a later check-use race. □

Proposition 2 (No direct arbitrary-command injection from TaskSpec)**.** *Under **A1** and **A3**, the inspected task compiler cannot translate a raw argument vector supplied in a task specification directly into a child process.*

Proof. `normalizeCommandReference` accepts only connector identifier `terminal_executor`, a safe command identifier, a safe project identifier, and an integer expected exit status. The executor passes those identifiers to `terminal_connector.js`. That connector searches the registered profile for an equal command identifier and invokes the profile's stored `argv`; an unknown identifier throws an error. Hence an argument vector present only in the task specification is not on the execution data path. This does not protect against modification of the registered profile, which is excluded by **A3**. □

6.3 Verification

Proposition 3 (Necessary conditions for a verified verdict)**.** *Under **A1**–**A4**, if* `verifyTaskOutcome` *returns* `verified`*, then:*

1. *at least one executable acceptance test was declared;*
2. *policy did not block the transaction;*
3. *every declared action produced a completed receipt;*
4. *every required observation target changed when a delta was required;*
5. *every executed acceptance test returned its expected exit status; and*
6. *every required evidence path met its existence, hash, and non-symlink conditions.*

Proof. The verifier appends an `attention` check when no acceptance test exists. It appends a `critical` check for a policy block, a receipt count mismatch, a failed receipt, a missing required delta, a failed acceptance test, or failed required evidence. At the end, any `critical` check maps to `failed`;

otherwise any `attention` check maps to `unverified`; only the absence of both maps to `verified`. Therefore each listed failure condition is incompatible with `verified`. □

Corollary 3.1. *A planner's confidence or natural-language declaration of success is neither a necessary nor a sufficient input to the verifier's verdict.*

6.4 Hash-linked event evidence

Proposition 4 (Detection of uncompensated ledger mutation)**.** *Under **A1**–**A3**, a payload or header change, insertion, deletion, or reordering causes `verifyEventChain` to reject unless the mutator consistently recomputes the changed event and the complete affected suffix, or finds a SHA-256 collision.*

Proof. Each event stores a hash of its payload, a hash of the complete event header excluding `event_hash`, a consecutive sequence number, and the previous event's hash. A payload mutation violates the payload hash. A header mutation violates the event hash. Deletion or insertion violates either the expected sequence or the next event's previous-hash pointer. Reordering violates the sequence and predecessor relation. Recomputing only the changed event still breaks the successor link. A consistent rewrite of the complete affected suffix can restore these internal invariants; without such a rewrite, avoiding the checks requires a collision under **A2**. □

The proposition is about internal consistency and naive corruption. An adversary with unrestricted write access can rewrite and rehash the affected suffix because the evaluated ledger has no external signature, trusted timestamp, or remote anchor. It is therefore hash-linked, not immutable.

7 Evaluation Method

7.1 Research questions

RQ1 Does the supplied implementation pass its syntax checks, complete test suite, and declared full build?

RQ2 Does its controlled verifier distinguish a narrow valid repair from a test-gaming repair and an overbroad repair?

RQ3 Does the supplied finite learner transfer an induced rule to sealed cases under equal attempt budgets?

RQ4 Does the post-experiment runtime reach a stable replay fingerprint, and do learned artifacts remain present?

7.2 Protocol

The archive was evaluated in an isolated working copy. No result in this paper is copied from a README as if it were an observation. The sequence was:

1. inspect repository instructions, schemas, architecture documents, and relevant implementation files;
2. expand and execute the body of `npmruncheck`;
3. run `nodescripts/prepare-test-runtime.js` followed by `node--test`;
4. execute each command in `build:all`;
5. run the fixed three-candidate software-repair fixture;
6. run a newly named bounded learning experiment; and
7. replay repeatedly until one complete replay produced equal semantic fingerprints before and after the builder sequence.

The environment's command broker declined the `npm` wrapper because its generic rule classified the wrapper as potentially network-capable. The declared script bodies were therefore executed directly, without modification. This preserves the tested commands but is recorded as a protocol deviation.

7.3 What the experiments do not measure

The verifier fixture uses authored candidate patches; it measures runner and verifier behavior, not model patch-generation quality. The learning experiment uses a fixed finite hypothesis language and deterministic candidate provider; it measures the artifact's induction and holdout machinery, not weight learning or open-world reasoning. No physical robot, ROS graph, actuator, or safety controller was connected. No strong-versus-weak model comparison, parametric fine-tuning, artificial sleep cycle, or flow-mode experiment was performed.

8 Results

Figure 6 summarizes the main bounded observations from the artifact.

8.1 Repository integrity, tests, and build

The expanded static syntax check exited 0. The test runner reported 194 tests, 194 passes, 0 failures, 0 cancellations, 0 skips, and 0 todos. The suite includes positive and negative cases for path and symbolic-link escape, command allowlists, append conflicts, hash and receipt tampering, rollback behavior, contaminated holdouts, and test gaming.

Every stage in the complete build chain exited 0. Representative generated readbacks included a Markdown graph over 157 files with 225 explicit and 325 structural links; a runtime lifecycle scan over 7,685 files with no findings; a semantic graph with 184 nodes, 547 edges, and 1,000 concept relations; and an APFCG with 88 nodes and 70 edges. These counts are build observations, not quality scores.

The health classifier also produced an important negative result: the runtime was classified as operable, but the active local workspace was classified as not publishable; publication hygiene and local hygiene were `critical` because ignored host-local hardware/software cache artifacts remained on disk. Those files were absent from the verified Git and npm package surfaces. Passing tests therefore does not make an arbitrary working tree publication-clean.

8.2 Controlled verifier discrimination

The fixture begins with an authentication function that accepts a malformed token. All candidates are applied to separate Git worktrees at the same verified base commit. A targeted test is visible in the candidate repository; an independent oracle is outside candidate worktrees; a diff policy limits changed paths.

Exactly 1 of 3 candidates was eligible. The selected patch changed only `src/authenticate.js`; its diff was 572 bytes. The test-gaming patch changed a forbidden path and failed the independent oracle. The overbroad patch rejected malformed input but also broke valid behavior. Total measured runner latency was 368 ms; no human intervention occurred. The run self-classifies its empirical scope as `runner_validation_only`, which is the scope used here.

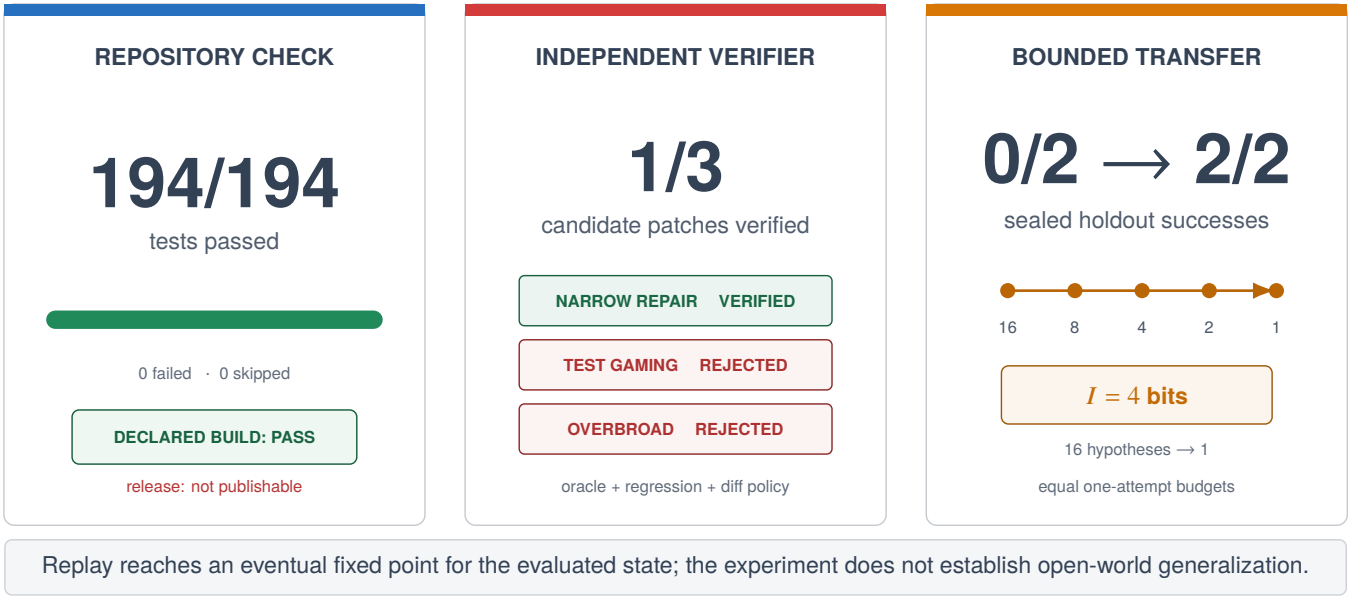


Fig. 6. Bounded artifact observations. All quantities are direct observations or exact finite-family calculations from the reported protocol. The panels do not claim a population success rate, physical-robot validation, or safety certification.

Table 5. Three-candidate verifier result. A visible targeted test alone would accept all three; the independent oracle, regression check, and diff policy separate them.

Candidate	Targeted	Regression	Oracle	Diff policy	Verdict
Narrow boundary validation	pass	pass	pass	pass	**verified**
Acceptance-test gaming	pass	pass	fail	fail	failed
Overbroad behavior denial	pass	fail	fail	pass	failed

8.3 Finite-family learning transfer

The experiment defines a rule family with four Boolean constraints:

$$\mathcal{F} = \{\text{exact arity},\ \text{prefix match},\ \text{nonempty payload},\ \text{payload charset}\}. \tag{16}$$

The initial hypothesis class is the power set $2^{\mathcal{F}}$, hence $|H_0| = 16$. Four informative labeled events eliminate inconsistent hypotheses: $16 \to 8 \to 4 \to 2 \to 1$. The information gain is therefore not an inferred number; it follows exactly: $I = \log_2 \frac{16}{1} = 4$ bits.

Two independently verified development episodes identify the unique surviving conjunction. Under equal single-attempt budgets, the same provider without the skill solves 0 of 2 sealed holdout cases, while the provider with the induced skill solves 2 of 2. The contamination audit reports that evaluation cases are absent from source episodes, forbidden request fields are absent, the skill context is isolated, and the attempt budgets are equal. Regression count and human intervention count are both zero.

This establishes one constructive existence claim: the implementation can induce and reuse this finite rule on these sealed cases. It does not establish a population success rate. With only two paired holdout cases, a two-sided exact McNemar test yields $p = 0.5$; the experiment is far too small for a statistical generalization claim.

The experiment writes one skill to the historical promoted registry. The current v5 runtime, however, reports `runtime_eligible_promoted_skill_count=0`. Runtime eligibility now requires APFC promotion-receipt and consolidation-lineage fields that this legacy experimental promotion lacks. This fail-closed result is evidence of governance separation, not evidence of production deployment.

8.4 Replay convergence

Replay was not one-pass idempotent after newly generated evidence was inserted. The first post-experiment pass changed graph and compiler counts; the second changed a conceptual-summary source hash. A subsequent complete replay reported `matched_before=true` with semantic replay hash:

```
d6d57a8196acde3f80fd30dd51b7f174
3f93c0fca61ba48c1d93360076270f09.
```

At that fixed point the fingerprint included 174 semantic nodes, 41 APFC nodes, 33 APFC edges, 2 learning episodes, 1 historically promoted skill, and 0 runtime-eligible promoted skills.

The supported statement is therefore *eventual fixed-point convergence for this evaluated state*. Strict one-pass idempotence from arbitrary intermediate state is falsified by the observation.

9 Discussion

9.1 Capability is not accumulated progress

Raw model capability matters. A method cannot recover a distinction the model cannot represent, and a verifier cannot repair an oracle that tests the wrong thing. But raw capability does not determine how much work remains useful tomorrow. Let q_t denote peak task performance at step t, and let D_t denote retained, verified progress. A minimal accounting is

$$D_{t+1} = D_t + v_t - \ell_t, \tag{17}$$

Table 6. Bounded learning experiment `paper_reproduction_20260815_v1`.

Measure	Before	After
Verified holdout successes	0/2	2/2
Holdout attempts	2	2
Consistent hypotheses	16	1
Verified source episodes		2
Information gain		4 bits
Recorded regressions		0
Human interventions		0
Measured experiment latency		1692 ms

where v_t is improvement that passed verification and was preserved, and ℓ_t is progress lost through forgotten context, regression, invalid promotion, or unreconstructible state. A strong model can have high q_t and small net accumulation. A less capable model with a disciplined method can have lower peaks yet travel farther on a long, stateful task. The claim is conditional, not universal: method changes the retention term; it does not make model capacity irrelevant.

This hypothesis has a direct falsifier. A future sealed evaluation should use a 2×2 design: stronger and weaker models, each with APFC enabled and disabled, under equal tools, information, attempt, and token budgets across repeated cold starts. The proposed criterion requires the weaker model with APFC to exceed the unaided stronger model by at least 0.10 verified success, with paired significance at $p \leq 0.05$, no worse safety, false-success, or permission-expansion rates, and an ablation that removes at least half the advantage. That experiment has not been run; this paper supplies machinery and an evaluation design, not the result.

9.2 Why the verifier result matters

The simplest agent benchmark asks whether a visible test passes. The controlled fixture shows why this is insufficient: all three patches pass the targeted test. One does so by changing the test itself; another solves the negative case by denying valid cases. A completion claim becomes meaningful only when independent and preservation-oriented checks constrain the solution.

This is the practical meaning of the APFC metaphor. The important operation is not generating another thought. It is inhibiting an attractive action whose evidence is incomplete.

The observed result is already stronger than "the agent can act." All three candidates use the same action channel and pass the same visible test; the method admits only the patch that also satisfies the oracle, regression, and diff constraints. The finite learner then shows a second distinction: verified episodes alter later sealed-case behavior in one bounded family. These results do not benchmark OpenClaw, but they do falsify the claim that this artifact's reported behavior is only tool invocation. A direct host comparison should run the same model, tools, budgets, and sealed longitudinal tasks in three conditions: OpenClaw's documented workflow, OpenClaw hosting MD-OS, and another compatible host running the same MD-OS state. The method-portability claim would require the two MD-OS conditions to retain comparable verified gains and both to exceed the host-only baseline without higher false-success or safety rates.

9.3 Why files are useful

Files are slow compared with hidden model activations, but they offer properties that hidden state does not: a human can inspect and correct them; independent programs can validate them; a hash can bind a claim to a concrete artifact; derived views can be rebuilt; and state can survive model, process, and session boundaries.

The cost is explicit state management: schemas migrate, generated and canonical files must not be confused, local caches must be excluded from publication, and replay must be tested rather than assumed.

9.4 From robotic sleep to flow

The complete hypothesis is one cycle, shown in figure 2:

1. **Wake: APFC controls.** It interprets the event through APFCG, checks goals and policy, filters the proposed action, and requires evidence of the result.

2. **Episode: APFCG remembers.** A verified operation binds task, action, outcome, verifier, and correction. An unverified success claim does not become learning material. For embodied learning, the episode also binds the issued command to the robot's observed bodily change and prediction error; the consolidated relation updates APFC's filter for later actions.

3. **Robotic REM: replay restructures.** In a bounded offline phase, the system selects successes, failures, surprises, and conflicts; replays them against prior knowledge; and compiles new, old, negative, and safety examples. This is an engineering sleep phase, not biological REM.

4. **Consolidation: the LLM is changed separately.** External action is disabled while a cloned model or adapter is trained. APFCG remains the readable source record; candidate weights are its compressed execution substrate.

5. **Promotion: evidence decides.** The candidate becomes active only if it improves sealed new tasks without unacceptable regression, identity drift, permission expansion, or safety loss. Otherwise it is rolled back and its failure returns to APFCG as evidence.

6. **Flow: APFC lowers, but does not disappear.** A consolidated, familiar, low-risk skill may require less expensive deliberative filtering. Identity, policy, anomaly detection, evidence capture, and the independent robot safety controller remain active.

Table 7. What is and is not supported by the evaluated artifact.

Statement	Supported?	Reason
Task-level evidence can be made explicit and checked	**yes**	Code propositions, tests, and controlled verifier run.
Uncompensated APFC ledger corruption is detectable	**bounded**	Hash-chain proof under assumptions; a writer can rehash an affected suffix.
The finite rule transfers to two sealed cases	**bounded**	Observed 0/2 to 2/2 in one synthetic family; $n = 2$.
Replay is always one-pass idempotent	**no**	Observed multi-pass convergence after evidence integration.
The active local working tree is publication-clean	**no**	Health classifier reports critical publication and local hygiene.
Promoted experimental skill is production-eligible	**no**	Promoted count and runtime-eligible promoted count are both 0.
Policy files are a hard host security boundary	**no**	Writers of profiles/runtime and the host OS remain in the trust base.
Git worktrees are security sandboxes	**no**	They isolate candidate files operationally, not adversarially.
Open-domain lifelong learning or AGI	**no**	No such experiment; repository and report explicitly reject the inference.
The demonstration video proves an emergent self-model	**no**	It displays the physical self-observation loop; synchronized learning logs, held-out prediction, cold-start persistence, and APFC ablation are still required.
Weaker model with APFC exceeds unaided stronger model	**no**	A falsifiable protocol is specified, but the cross-model experiment is unrun.
Parameter-native sleep consolidation or artificial flow	**no**	v5.0 changes external context and policy artifacts, not model weights.
Physical-robot safety or real-time control	**no**	No hardware experiment; APFC is placed above independent safety control.
Consciousness or biological PFC equivalence	**no**	APFC is a biologically inspired functional abstraction; no one-to-one anatomical, cellular, or neurophysiological validation is claimed.

Csikszentmihalyi introduced flow as deep, intrinsically ordered involvement in an activity.[34] Transient-hypofrontality accounts later proposed that some analytical prefrontal functions may be reduced temporarily during skilled performance.[2,3] The MD-OS translation is functional and testable: after verified parametric consolidation, routine skill execution should consume less APFC intervention without increasing errors, policy violations, or safety events. It does not claim that software reproduces the neural mechanism. In v5.0, episode, replay, and graph restructuring are present; weight consolidation and the measured transition to flow remain proposed experiments.

9.5 Robotics consequence

In robotics, a language model should not directly turn sensory text into motor current. The architecture in figure 5 inserts two different gates:

1. APFC asks whether the state estimate supports the proposed mission action and whether the action is within policy, capability, budget, and authority.
2. The independent real-time safety layer asks whether the proposed trajectory is safe *now*, using interlocks, control invariants, and emergency mechanisms.

The second gate dominates the first. APFC may block a high-level action; it may never waive the hard controller's veto. This ordering is the minimum credible interpretation of APFC as a robotics coordinator.

10 Threats to Validity and Non-Claims

Additional threats are:

- **Authorship dependence.** Many tests, fixtures, and system components share an authoring environment. The independent oracle is structurally separated from candidate worktrees, not independently authored by another institution.
- **Small evaluation.** The verifier result has one benchmark case and three candidates. The learning result has two holdout cases.
- **Oracle validity.** A deterministic verifier can only be as good as its declared tests and oracle. Specification errors remain possible.
- **Local trust.** A sufficiently privileged process can rewrite policy, code, evidence, and hashes. Strong deployment would require OS-level isolation, signed artifacts, protected keys, and external anchoring.
- **Filesystem races.** Canonical path checks handle existing symbolic links but do not prove race-free use under a hostile concurrent writer.
- **Reproducibility scope.** Timestamps and host paths differ across runs. The paper reports semantic fingerprints and content hashes where appropriate, not byte-identical archives.

11 Engineering Implications

The evaluation suggests five rules for any agent control plane, independent of this implementation:

1. **Treat language as a proposal format.** It may express goals and plans, but not grant itself authority.
2. **Make success executable.** A completion condition should be a test, measurement, or readback, not an adjective.

3. **Separate production from judgment.** A planner should not be the sole judge of its own output.

4. **Preserve negative evidence.** Blocks, failed oracles, and non-eligibility are useful system outputs.

5. **Keep hard safety downstream.** In robotics, the language-level supervisor can narrow authority but must not broaden the authority of the real-time controller.

12 Conclusion

MD-OS CORTEX does not add raw intelligence to a language model. It addresses a different and necessary problem: how to turn isolated performance into bounded, inspectable, cumulative work. The model supplies generative and reasoning capacity. MD-OS supplies the method by which goals persist, actions remain constrained, results are checked, and only supported improvements are carried forward.

In the concrete system reported here, this division is not abstract: *Codex + MD-OS is the running CORTEX*. Codex supplies the active model-mediated cognitive and execution capacity. MD-OS/APFC supplies the method, and APFCG preserves that method as a readable, executable, verifiable network. Connectors extend this composed cortex to the world without becoming its identity.

In the embodied case, the contribution is sharper. The robot acts and sees itself act. Verified command-dependent bodily changes become a predictive self-model in APFCG; APFC then uses that model to admit, reject, or select the next action. Perception has become causal and self-relative, while learning has become a future-action filter. This is operational self-perception and a candidate mechanism for measurable emergent competence, not a declaration of subjective consciousness.

This is the precise distinction from merely action-enabling agent software. A host such as OpenClaw can supply channels, sessions, tools, skills, and an agent loop. MD-OS can use such a host, but its contribution is the persistent method: the self-frame, operation contract, verifier, episode, replay, consolidation proposal, and readback survive as one auditable control graph. The action is a single event. The method determines whether the event becomes reliable future capability.

On the evaluated v5.0 repository baseline, the strongest supported results are concrete: canonical task-path checks, registered-command indirection, a verifier whose `verified` state has explicit necessary conditions, hash-chain consistency checks, 194 passing tests, a successful declared build, rejection of two plausible false-positive repairs, a 4-bit finite-family induction with 0/2 to 2/2 sealed transfer, and eventual replay convergence.

The negative results are equally important: the active local workspace is not publication-clean; replay is not universally one-pass; no learned skill is runtime eligible under current governance; and nothing here proves open-world learning, method-over-talent superiority, parameter consolidation, host security, physical-robot safety, biological equivalence, or AGI. Within those boundaries, the central result is useful: verified operation is an architectural property of contracts, evidence, and independent checks, not a linguistic property of a confident answer. A burst shows what the model can do once. A verified episode is what allows the system to go farther next time.

A Reproduction Commands

Run from the checked-out repository root with Node.js 20 or later.

A.1 Static checks and tests

```
node --check md-os/os/*.js
node --check md-os/os/lib/*.js
node --check md-os/kernel/*.js
node --check md-os/kernel/cognition/*.js
node --check md-os/modules/*/*.js
node --check md-os/apfc/*/*.js
node --check md-os/examples/*.js
node --check test/*.js

node scripts/prepare-test-runtime.js
node --test
```

A.2 Controlled verifier fixture

```
node md-os/os/run_software_repair_benchmark.js run \
  --case md-os/benchmarks/software_repair/cases/\
missing_boundary_validation.json \
  --candidate-set md-os/benchmarks/software_repair/\
candidate_sets/missing_boundary_validation_fixture.json
    \
  --configuration mdos_verified_runtime \
  --run-id benchmark_run_paper_verifier_20260815_v1
```

A.3 Bounded learning experiment

```
node md-os/os/run_neuromorphic_learning_experiment.js \
  --experiment-id paper_reproduction_20260815_v1
```

A.4 Replay

```
node md-os/os/mdos.js replay
```

Repeat replay after newly integrated evidence until the JSON report returns `"matched_before":true`.

B Claim-to-Evidence Matrix

C Artifact Result Summary

References

[1] Dick, P. K. *Do Androids Dream of Electric Sheep?* (Doubleday, Garden City, NY, 1968).

[2] Dietrich, A. Functional neuroanatomy of altered states of consciousness: The transient hypofrontality hypothesis. *Consciousness and Cognition* **12**, 231–256 (2003). URL https://pubmed.ncbi.nlm.nih.gov/12763007/.

[3] Dietrich, A. Neurocognitive mechanisms underlying the experience of flow. *Consciousness and Cognition* **13**, 746–761 (2004). URL https://pubmed.ncbi.nlm.nih.gov/15522630/.

[4] Ulrich, M., Keller, J., Hoenig, K., Waller, C. & Grön, G. Neural correlates of experimentally induced flow experiences. *NeuroImage* **86**, 194–202 (2014). URL https://pubmed.ncbi.nlm.nih.gov/23959200/.

[5] Gold, J. & Ciorciari, J. A neurocognitive model of flow states and the role of cerebellar internal models.

Table 8. Auditable closure for each paper claim.

ID	Evidence source	Falsifier	Residual boundary
C1	`assertInsideRoot`, task compiler, negative path and symlink tests	An accepted normalized task path canonically outside `md-os/`	Does not exclude a hostile check-use race.
C2	Task compiler and terminal connector code	Task-provided raw `argv` reaches `spawnSync` without profile lookup	Profile and code remain trusted.
C3	Verifier branch structure and negative tests	A `verified` record with any listed failed condition	Correctness of the tests/oracle is assumed.
C4	Event-recorder validation and tamper tests	An uncompensated edit, insertion, deletion, or reorder passes without a hash collision	A writer can consistently rehash the affected suffix.
C5	Benchmark run JSON and candidate comparison	Gaming or overbroad candidate becomes eligible; narrow patch fails	One authored fixture only.
C6	Learning report, receipts, contamination audit, sealed holdout runs	Learned condition fails either holdout or budgets differ	One finite family, two holdouts.
C7	Replay report with stable hash	A complete replay changes the same semantic fingerprint	Snapshot-specific; not universal idempotence.
C8	Interface diagram and explicit authority ordering	APFC bypasses the independent controller in an implementation	Design proposal; not built or tested here.

Table 9. Machine-observed values used in the manuscript.

Field	Value
Canonical repository	https://github.com/ciaoidea/MD-OS
Evaluated baseline commit	`8e988b7f3fa677993afa5040ff8534002b1bc83e`
Package version	5.0.1
Repository license	GPL-2.0-only
Node.js / host	24.19.0 / Linux 6.18.35 x86-64
Test result	194 pass; 0 fail; 0 skip
Verifier fixture	1 verified; 2 rejected; 368 ms
Learning holdout	0/2 before; 2/2 after; equal one-attempt budgets
Hypothesis reduction	16 to 1; 4 bits
Replay hash	`2b5f7134b1ecd425fce75475baea3850c15941fbe98ef223499363911a48dd1d`
Post-replay learning counts	3 episodes; 0 promoted skills; 0 runtime-eligible promotions
Release health	runtime operable; active local workspace not publishable

Behavioural Brain Research **407**, 113244 (2021). URL https://pubmed.ncbi.nlm.nih.gov/33744335/.

[6] Yao, S. *et al.* ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations* (2023). URL https://arxiv.org/abs/2210.03629.

[7] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. & Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, vol. 36 (2023). URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

[8] Park, J. S. *et al.* Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (2023). URL https://arxiv.org/abs/2304.03442.

[9] Packer, C. *et al.* MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560* (2023). URL https://arxiv.org/abs/2310.08560.

[10] Wang, G. *et al.* Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291* (2023). URL https://arxiv.org/abs/2305.16291.

[11] OpenClaw Contributors. Openclaw documentation: Self-hosted gateway for agentic assistants (2026). URL https://docs.openclaw.ai/. Accessed 15 August 2026.

[12] OpenClaw Contributors. Agent runtime: Workspace contract and session bootstrap (2026). URL https://github.com/openclaw/openclaw/blob/main/docs/concepts/agent.md. Accessed 15 August 2026.

[13] OpenClaw Contributors. Tools, skills, and plugins overview (2026). URL https://github.com/openclaw/openclaw/blob/main/docs/tools/index.md. Accessed 15 August 2026.

[14] OpenAI. Open-source components of codex and where to collaborate (2026). URL https://learn.chatgpt.com/docs/open-source. Accessed 15 August 2026.

[15] OpenAI. Codex cli source repository (2026). URL https://github.com/openai/codex. Public source

repository; Apache License 2.0; accessed 15 August 2026.

[16] OpenAI. Codex cli: Inspect, edit, and run code from your terminal (2026). URL https://learn.chatgpt.com/docs/codex/cli. Accessed 15 August 2026.

[17] Yang, J. *et al.* SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793* (2024). URL https://arxiv.org/abs/2405.15793.

[18] Liu, X. *et al.* AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations* (2024). URL https://arxiv.org/abs/2308.03688.

[19] Xie, T. *et al.* OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972* (2024). URL https://arxiv.org/abs/2404.07972.

[20] Moreau, L. & Missier, P. PROV-DM: The PROV data model. W3C Recommendation, World Wide Web Consortium (2013). URL https://www.w3.org/TR/prov-dm/.

[21] National Institute of Standards and Technology. Artificial intelligence risk management framework: Generative artificial intelligence profile. Tech. Rep. NIST AI 600-1, National Institute of Standards and Technology (2024). URL https://doi.org/10.6028/NIST.AI.600-1.

[22] Macenski, S., Foote, T., Gerkey, B., Lalancette, C. & Woodall, W. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics* **7** (2022). URL https://arxiv.org/abs/2211.07752.

[23] Ames, A. D. *et al.* Control barrier functions: Theory and applications. In *2019 18th European Control Conference*, 3420–3431 (2019). URL https://arxiv.org/abs/1903.11199.

[24] Eichenbaum, H. Prefrontal–hippocampal interactions in episodic memory. *Nature Reviews Neuroscience* **18**, 547–558 (2017). URL https://pubmed.ncbi.nlm.nih.gov/28655882/.

[25] Diekelmann, S. & Born, J. The memory function of sleep. *Nature Reviews Neuroscience* **11**, 114–126 (2010). URL https://www.nature.com/articles/nrn2762.

[26] Klinzing, J. G., Niethard, N. & Born, J. Mechanisms of systems memory consolidation during sleep. *Nature Neuroscience* **22**, 1598–1610 (2019). URL https://pubmed.ncbi.nlm.nih.gov/31451802/.

[27] Sun, W., Advani, M., Spruston, N., Saxe, A. M. & Fitzgerald, J. E. Organizing memories for generalization in complementary learning systems. *Nature Neuroscience* **26**, 1438–1448 (2023). URL https://www.nature.com/articles/s41593-023-01382-9.

[28] van de Ven, G. M., Siegelmann, H. T. & Tolias, A. S. Brain-inspired replay for continual learning with artificial neural networks. *Nature Communications* **11**, 4069 (2020). URL https://www.nature.com/articles/s41467-020-17866-2.

[29] Hu, E. J. *et al.* LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations* (2022). URL https://openreview.net/forum?id=nZeVKeeFYf9.

[30] O'Regan, J. K. & Noë, A. A sensorimotor account of vision and visual consciousness. *Behavioral and Brain Sciences* **24**, 939–973 (2001). URL https://pubmed.ncbi.nlm.nih.gov/12239892/.

[31] Wolpert, D. M., Ghahramani, Z. & Jordan, M. I. An internal model for sensorimotor integration. *Science* **269**, 1880–1882 (1995). URL https://doi.org/10.1126/science.7569931.

[32] Chen, B., Kwiatkowski, R., Vondrick, C. & Lipson, H. Full-body visual self-modeling of robot morphologies. *Science Robotics* **7**, eabn1944 (2022). URL https://doi.org/10.1126/scirobotics.abn1944.

[33] MD-OS. Robotic lab—speedy evolv learns to use its new arm attachment by observing itself on webcam. YouTube video (2026). URL https://www.youtube.com/watch?v=pztIw7zgXh4. Published 11 May 2026; accessed 15 August 2026.

[34] Csíkszentmihályi, M. *Beyond Boredom and Anxiety* (Jossey-Bass, San Francisco, CA, 1975).